

POINTERS

Lecture: 13,14,15

Reference: Chapter 6 (TYC), Chapter 8,9 (LUC)

POINTER

- A pointer is a variable
 - **Holds the memory address of an object**
 - Address are whole numbers/integer
 - So pointers should contain only whole numbers
- A variable **p** contains address of another variable **q**
 - **p** is said to “point to” **q**
 - Let **q** is at location 1017 in memory, then **p** would have the value 1017
- General form
 - `type *var_name;`
 - *type* is the *base type* of the pointer
 - *base type*: **type of the object that the pointer can point to**
- Example: `int *p;` // holds the location of a memory location where an integer is stored

POINTER OPERATORS

- Two special operators: * and &
- * operator (at address)
 - Indirection operator/dereferencing operator
 - **Returns the value stored at the address it precedes**
 - No relation to the multiplication operator
 - Let **p** is a pointer, points to **q**. If **q** contains value 67
 - `printf("%d\n", *p);`
 - Will print **67**

POINTER OPERATORS

- Two special operators: * and &
- & operator (address of)
 - **Returns the address of the variable it precedes**
 - Can not applied to constants or expressions
 - Let **p** is a pointer, points to **q**.
 - **p=&q;**
 - Assigns the address of **q** to **p**

POINTER OPERATORS

- Two special operators: * and &
- & and * are complements of each other
- Printing *(&i) is same as printing i
- Applying both consecutively in either result yields same result
 - &*p and *&p gives same result (check yourself!)

POINTER

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int *p, q;
```

```
    q=197;
```

```
    p=&q;
```

```
    printf("%d\n", *p);
```

```
    return 0;
```

```
}
```

Assigns **p** the address of **q**

- Prints the value at address **p**
- Variable's value is referenced through a pointer
- The process is called *indirection*

POINTER DECLARATION

- `int *p;`
 - A pointer to an integer value
- `int *p, i;`
 - `i` is an integer variable (not a pointer)
 - The indirection operator (*) does not distribute to all variable names in the declaration
 - Each pointer must be declared with the * prefixed to the name (`int *p,*i;`)
- `float *fp;`
 - Does not mean **fp** will contain floating point value
 - **fp will contain address of a floating point value**

POINTER

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int *p, q;
```

```
    p=&q;
```

```
    *p=197;
```

```
    printf("%d\n", q);
```

```
    return 0;
```

```
}
```

- Possible to use * operator at the left side of an assignment operation
- Assigns value indirectly

POINTER

- Let **q** be located at address 210 and **p** is located at address 208

```
int *p, q;
```

```
p=&q;
```

```
*p=1046;
```

POINTER

- Let **q** is located at address 210 and **p** is located at address 208

```
int *p, q;
```

- Now **p** and **q** contains nothing/garbage value

Location	Contents
208	Unknown
210	Unknown

POINTER

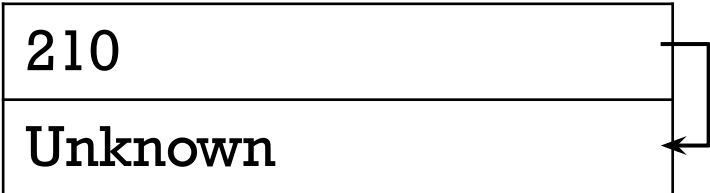
- Let **q** is located at address 210 and **p** is located at address 208

```
int *p, q;
```

```
p=&q;
```

- **p** contains the value 210

Location	Contents
208	210
210	Unknown



p points to
q

POINTER

- Let **q** is located at address 210 and **p** is located at address 208

```
int *p, q;
```

```
p=&q;
```

```
*p=1046;
```

- **q** contains the value 1046
- **p** has nothing to do with the value of **q**
- It holds the address of **q**

Location

Contents

208

210

210

1046

p points to
q

POINTER

- C allows any type of pointer to point anywhere in the memory
- Base type is very important!!
 - **Determines how many bytes are in the object pointed to**

POINTER

```
int q;  
double *fp;  
fp=&q;  
*fp=100.23;
```

- Syntactically correct but **wrong**
- **int** shorter than **double**
- For example: integer takes 2 bytes of memory where double takes 8 bytes
- The assignment statement uses the 2 bytes allocated to **q**
- **As well as it uses 6 adjacent bytes (which are not declared for q !!)**
 - Causing an error

POINTER

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int *p;
```

```
    double q, temp;
```

```
    temp=1234.34;
```

```
    p=&temp;
```

```
    q=*p;
```

```
    printf("%f\n", q);
```

```
    return 0;
```

```
}
```

- **p** is an integer pointer
- Cannot be used to transfer 8-byte quantity
- Only 2 bytes transferred

POINTER

- Using a pointer before assigning: program may crash
- Declaring a pointer variable creates a variable capable of holding a memory address
- No meaningful initial value is given

```
int main()
{
    int *p;
    *p=197;
    return 0;
}
```

- **Incorrect: `p` is not pointing to anything**

POINTER

```
int main()
{
    int *p, q;
    p=&q;
    printf("%p\n",p);
    return 0;
}
```

- %p conversion specifier is used to output the value of a pointer
- %p outputs memory location as a hexadecimal integer

POINTER EXPRESSION (ARITHMETIC EXPRESSION)

- Operators:
 - ++
 - +
 - --
 - -
- Only integer quantities can be added or subtracted
- **multiplication, division, modulus operators can not be applied**

POINTER EXPRESSION (ARITHMETIC EXPRESSION)

- Performed relative to base type of the pointer
- Each increment will cause the pointer to **point to the next item**
- **Example:** an integer pointer **p** contains the address 200
 - Assuming integers are 2 bytes long
- After **p++**, **p** will have the value 202
- **p=p+204** causes **p** to point to the 204th integer past one pointing
 - The value of **p** will be $200+2*204$ or 608

POINTER EXPRESSION (ARITHMETIC EXPRESSION)

- Performed relative to base type of the pointer
- Each increment will cause the pointer to point to the next item

```
float *fp, f;  
fp=&f;  
*fp=3.14;
```

Let address of **fp**
& **f** is 3140 & 5682
respectively and
float is 4 bytes

Location

Contents

3140

5682

3142

Unknown

...

5682

3.14

5686

Unknown

fp points
to **f**

POINTER EXPRESSION (ARITHMETIC EXPRESSION)

- Performed relative to base type of the pointer
- Each increment will cause the pointer to point to the next item

```
float *fp, f;  
fp=&f;  
*fp=3.14;  
fp++;
```

Location

Contents

3140

5686

3142

Unknown

...

5682

3.14

5686

Unknown

After `fp++;`
fp points to the next item

POINTER EXPRESSION (ARITHMETIC EXPRESSION)

- **Subtraction**
- Example: an integer pointer **p** contains the address 200
 - Assuming integers are 2 bytes long
- $p = p + 204$ causes **p** to point to the 204th integer past one pointing
 - The value of **p** will be 608($200 + 2 * 204$)
- Now, $p = p - 204$ would set **p** back to 200
- One pointer can be subtracted from other
 - **Number of elements separating them can be found**
 - Meaningless unless performed on an array

POINTER EXPRESSION (ARITHMETIC EXPRESSION)

- Operators can be applied **to the pointer** or **the object to which it points**
- ***p++**: **increments p** (p now points to the next element)
- **(*p)++** : increments the value stored at **p**

POINTER EXPRESSION (COMPARISON & ASSIGNMENT EXPRESSION)

- Two pointers can be compared using relational operators
- Makes sense only if points to the same type of object
- Gives info about array indices of the pointed elements
- Can be compared to zero or NULL
- A pointer can be assigned to another pointer of the same type

POINTER WITH ARRAYS

- Array name gives the base address of the array
- Array name always points to the beginning of the array
- An array name is **a constant pointer**
- **Pointer is a variable but the array name is not**
- Can not be modified by pointer arithmetic
- So, these expressions are invalid (from the previous example)
 - **num++**
 - **num+=3**

POINTER WITH ARRAYS

- Array name gives the **base address** of the array
- Base address: address of the first element/ element at index zero
- So ***num/ *(num+0)** gives the value of the first element
- **num[i]**: compiler converts it to ***(num+i)**
- The following notations are same
 - **num[i]**
 - ***(num+i)**
 - ***(i+num)**
 - **i[num]**

POINTER WITH ARRAYS

```
int main()
{
    int num[]={10, 20, 30, 40, 50, 60, 70, 80};
    int *p, *q;
    p=&num[2];
    q=&num[5];
    printf("%d\n", q-p);
    return 0;
}
```

- Returns 3, not 6

POINTER WITH ARRAYS

```
int main()
{
    int num[]={10, 20, 30, 40, 50, 60, 70, 80};
    int *p, *q;
    p=&num[2];
    q=&num[5];
    printf("%d\n", p-q);
    return 0;
}
```

- Returns -3

POINTER WITH ARRAYS

- **Array name without index: a pointer to the start of the array**

```
int a[5]={1,2,3,4,5};
```

```
int *p;
```

```
p=a;
```

```
printf("%d %d %d\n", *p, *(p+1), *(p+2));
```

```
printf("%d %d %d\n", a[0], a[1], a[2]);
```

- ***(indirection operator) has a higher precedence then +(addition operator) so parentheses are necessary**

POINTER WITH ARRAYS

```
int main()
{
    int num[]={1, 2, 3, 4, 5};
    for(int i=0; i<5; i++)
    {
        printf("%d. value=%d, address=%d\n", i, num[i], &num[i]);
    }
    return 0;
}
```

Output:

```
0. value=1, address=1638196
1. value=2, address=1638200
2. value=3, address=1638204
3. value=4, address=1638208
4. value=5, address=1638212
```

- Array elements are stored in **contiguous** memory location
- Address output will be in **hexadecimal** if **%p** is used

POINTER WITH ARRAYS

```
int main()
{
    int num[]={1, 2, 3, 4, 5};
    int i, *j;
    j=&num[0];
    for(i=0; i<5; i++)
    {
        printf("%d. value=%d\n", i, *j);
        j++;
    }
    return 0;
}
```

Output:

```
0. value=1
1. value=2
2. value=3
3. value=4
4. value=5
```

POINTER WITH ARRAYS

```
int main()
{
    int num[]={1, 2, 3, 4, 5};
    int i, *j;
    j=num; /////
    for(i=0; i<5; i++)
    {
        printf("%d. value=%d\n", i, *j);
        j++;
    }
    return 0;
}
```

Output:

0. value=1
1. value=2
2. value=3
3. value=4
4. value=5

POINTER AND MULTIDIMENSIONAL ARRAY

```
int main()
{
    int s[3][2]={{1, 2}, {3, 4}, {5, 6}};
    for(int i=0; i<5; i++)
    {
        printf("address of %d th 1-d array: %d\n", i, s[i]);
    }
    return 0;
}
```

- If integer **size 4 byte**
- So each row takes 8 bytes
- So **s[1]** is interpreted as ***(s+1)**
- Address output will be in hexadecimal if **%p** is used

Output:

address of 0 th 1-d array: 1638192
address of 1 th 1-d array: 1638200
address of 2 th 1-d array: 1638208
address of 3 th 1-d array: 1638216
address of 4 th 1-d array: 1638224

POINTER AND MULTIDIMENSIONAL ARRAY

- Refer to the element `s[2][1]` using pointer
 - `s[2]` will give the address of first element of 3rd row (1638208)
 - $(1638208)+1$ will give the next address ,i.e. 1638212
 - Or `(s[2]+1)` will give 1638212

S[0][0]	S[0][1]	S[1][0]	S[1][1]	S[2][0]	S[2][1]
1	2	3	4	5	6
1638192	1638196	1638200	1638204	1638208	1638212

POINTER AND MULTIDIMENSIONAL ARRAY

- Value can be obtained by
 - `*(s[2]+1)`
 - `s[2]` is same as `*(s+2)`
 - So `*(s[2]+1)` is same as `*(*(s+2)+1)`
- So the following notations are same
 - `s[2][1]`
 - `*(s[2]+1)`
 - `*(*(s+2)+1)`

S[0][0]	S[0][1]	S[1][0]	S[1][1]	S[2][0]	S[2][1]
1	2	3	4	5	6
1638192	1638196	1638200	1638204	1638208	1638212

POINTER AND MULTIDIMENSIONAL ARRAY

- Accessing a multidimensional array using a pointer:

```
float gpa[10][5];
```

- To access **gpa[3][1]** using a pointer

```
float *fp;
```

```
fp=(float*)gpa;
```

```
float my_gpa=*(fp+(3*5)+1);
```

- The type of pointer generated by **gpa** is to a **two dimensional array** of floats
- **So cast is needed** (The compiler will show warning otherwise)

ARRAY OF POINTERS

- Pointer variable contains address
- Array of pointers is a collection of addresses

```
int main()
{
    int *pa[3];
    int a=97, b=21, c=54;
    pa[0]=&a;
    pa[1]=&b;
    pa[2]=&c;
    for(int i=0; i<3; i++)
    {
        printf("%d\n", *(pa[i]));
    }
    return 0;
}
```

ARRAY OF POINTERS

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a[]={1, 2, 3, 4, 5};
```

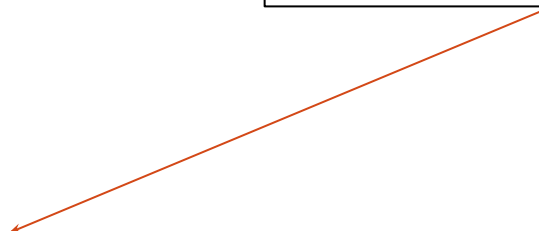
```
    int *pa[]={a, a+1, a+2};
```

```
    printf("%p %p %d\n", pa, *pa, *(*pa));
```

```
    return 0;
```

```
}
```

Multiple indirection



- It will print the base address of **pa**, the base address of **a**, and the value at the base address of **a** (which is **1**) respectively.

POINTERS AND STRING

```
int main()
{
    char str[]="CSE 161";
    char *p;
    p=str;
    for(int i=0; p[i]; i++){
        printf("%c", p[i]);
    }
    while(*p){
        printf("%c", *p++);
    }
    return 0;
}
```

Output:
CSE161CSE161

If the order of while loop and the for loop is exchanged the code will not work
Find out why?

POINTERS AND STRING

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    char *p;
```

```
    p="CSE 161";
```

```
    printf(p);
```

```
    return 0;
```

```
}
```

- Will the code work??
- YES!! Stores "CSE 161" in memory, then assigns its base address to p

STRING FUNCTIONS (LET US C: CHAPTER-9)

The most commonly used functions in the string library (string.h) are:

strlen – returns the length of a string

strcat - concatenate two strings

strchr - returns the pointer to the position of the first occurrence of a character

strcmp - compare two strings

strcpy - copy a string

strncat - concatenate one string with part of another

strncmp - compare parts of two strings

strncpy - copy part of a string

strrchr - returns the pointer to the position of the last occurrence of a character

STRLEN

- This function counts the number of characters present in a string.

```
#include<stdio.h>
#include<string.h>
```

```
main()
```

```
{
```

```
    char arr[ ] = "Bamboozled" ;
```

```
    int len1, len2 ;
```

```
    len1 = strlen ( arr ) ;
```

```
    len2 = strlen ( "Humpty Dumpty" ) ;
```

```
    printf ( "\nstring = %s ,length = %d", arr, len1 ) ;
```

```
    printf ( "\nstring = %s ,length = %d", "Humpty Dumpty", len2 ) ;
```

```
}
```

Output:

string = Bamboozled ,length = 10

string = Humpty Dumpty ,length = 13

- The function strlen() takes the base address of the string and returns the length of the string.
- While calculating the length it doesn't count '\0'.

X_STRLEN (USER DEFINED)

```
/* A look-alike/user defined of the function strlen() */
#include<stdio.h>
x_strlen ( char *s ){
    int length = 0 ;
    while ( *s != '\0' )
    {
        length++ ;
        s++ ;
    }
    return ( length ) ;
}
main(){
    char arr[ ] = "Bamboozled" ;
    int len1, len2 ;
    len1 = x_strlen ( arr ) ;
    len2 = x_strlen ( "Humpty Dumpty" ) ;
    printf ( "\nstring = %s ,length = %d", arr, len1 ) ;
    printf ( "\nstring = %s ,length = %d", "Humpty Dumpty", len2 ) ;
}
```

Output:

string = Bamboozled ,length = 10
string = Humpty Dumpty ,length = 13

STRCPY

- This function copies the contents of one string into another.
- The base addresses of the source and target strings should be supplied to this function.

```
#include<stdio.h>
#include<string.h>
```

```
main()
{
    char source[ ] = "Sayonara" ;
    char target[20] ;
    strcpy ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}
```

Output:

```
source string = Sayonara
target string = Sayonara
```

- On supplying the base addresses, strcpy() goes on copying the characters in source string into the target string till it doesn't encounter **the end of source string ('\\0')**.
- Target string's dimension should be big enough to hold the string being copied into it

X_STRCPY (USER DEFINED)

```
/* A look-alike of the function strcpy( ) */
#include<stdio.h>
x_strcpy ( char *t, char *s ){
    while ( *s != '\0' ){
        *t = *s ;
        s++ ;
        t++ ;
    }
    *t = '\0' ;
}
main(){
    char source[ ] = "Sayonara" ;
    char target[20] ;
    x_strcpy ( target, source ) ;
    printf ( "\nsource string = %s", source ) ;
    printf ( "\ntarget string = %s", target ) ;
}
```

Output:

source string = Sayonara
target string = Sayonara

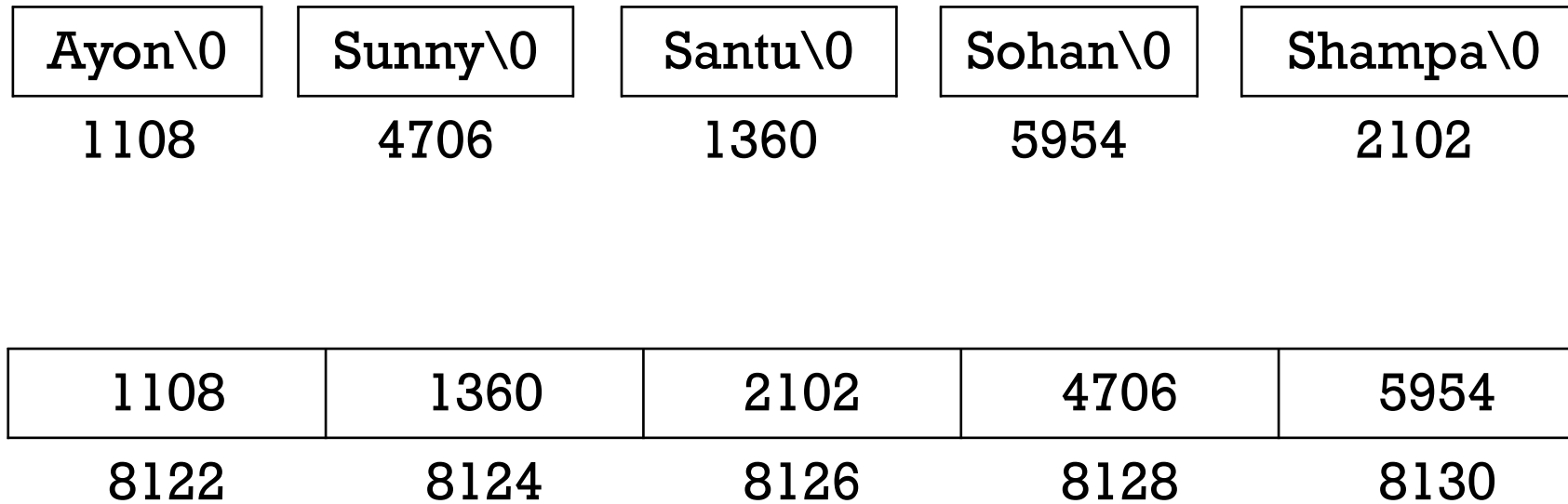
- It is necessary to place a '\0' into the target string, to mark its end.

ARRAYS OF POINTERS TO STRINGS

```
char *names[]={  
    "Ayon",  
    "Santu",  
    "Shampa",  
    "Sohan",  
    "Sunny"  
}
```

- **names[]** is an array of pointers
- Contains the **base address of respective names**

ARRAYS OF POINTERS TO STRINGS



- Note the change in the memory

ARRAYS OF POINTERS TO STRINGS

```
// prints out the names
int main()
{
    char *names[]={"Ayon", "Santu", "Shampa", "Sohan", "Sunny", ""};
    for(int i=0; *names[i]; i++)
    {
        printf("%s\n", names[i]);
    }
    return 0;
}
```


ARRAYS OF POINTERS TO STRINGS

```
// interchanging names (character by character)
int main()
{
    char names[][10]={"Ayon", "Santu", "Shampa", "Sohan", "Sunny"};
    char t;
    for(int i=0; i<10; i++)
    {
        t=names[2][i];
        names[2][i]=names[4][i];
        names[4][i]=t;
    }
    printf("%s %s\n", names[2], names[4]);
    return 0;
}
```

ARRAYS OF POINTERS TO STRINGS

```
// interchanging names (using pointer)
int main()
{
    char *names[]={"Ayon", "Santu", "Shampa", "Sohan", "Sunny"};
    char *temp;
    temp=names[2];
    names[2]=names[4];
    names[4]=temp;
    printf("%s %s\n", names[2], names[4]);
    return 0;
}
```

ARRAYS OF POINTERS TO STRINGS

- Limitations

```
int main()
{
    char *names[6];
    for(int i=0; i<6; i++)
        scanf("%s", names[i]);
    return 0;
}
```

- The program will crash
- As no valid address is contained

MULTIPLE INDIRECTIRECTION

```
int main()
{
    char **mp, *p, ch;
    p=&ch;
    mp=&p;
    **mp='A';
    printf("%c\n", ch);
    return 0;
}
```

mp			p			ch
7092	...	...	7541	...	...	'A'
5166			7092			7541

POINTER TO STRUCTURE

```
main()  
{  
    struct book  
    {  
        char name[25] ;  
        char author[25] ;  
        int callno ;  
    } ;  
    struct book b1 = { "Let us C", "YPK", 101 } ;  
    struct book *ptr ;  
    ptr = &b1 ;  
    printf ( "\n%s %s %d", b1.name, b1.author, b1.callno ) ;  
    printf ( "\n%s %s %d", ptr->name, ptr->author, ptr->callno ) ;  
}
```

Output:
Let us C YPK 101
Let us C YPK 101

POINTER TO STRUCTURE

```
/* Passing address of a structure variable */
struct book{
    char name[25] ;
    char author[25] ;
    int callno ;
};
main(){
    struct book b1 = { "Let us C", "YPK", 101 };
    display ( &b1 );
}
display ( struct book *b ){
    printf ( "\n%s %s %d", b->name, b->author, b->callno );
}
```

Output:
Let us C YPK 101

POINTER TO STRUCTURE

```
#include<stdio.h>
struct point {
    int x;
    int y;
};
int main(){
    struct point p3={0,0};
    struct point *pp;
    pp=&p3;
    pp->x=4;
    pp->y=8;
    printf("%d, %d\n", p3.x, p3.y);
    return 0;
}
```

Output:
4,8

Reverse an array using pointers

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
    int a[]={2,4,1,2,3};
```

```
    int size=5;
```

```
    int *pt=a;
```

```
    int temp;
```

```
    for(int i=0;i<size/2;i++)
```

```
    {
```

```
        temp=*(a+i);
```

```
        *(a+i)=*(a+size-1-i);
```

```
        *(a+size-1-i)=temp;
```

```
    }
```

```
    for(int i=0;i<size;i++)
```

```
        printf("%d ",a[i]);
```

```
}
```


Matrix multiplication using pointers

```
79 #include<stdio.h>
80
81 int main()
82 {
83     int a[4][4]={1,2,3,4,
84                 5,6,7,8,
85                 9,10,11,12,
86                 13,14,15,16};
87
88     int b[4][3]={1,2,9,
89                 3,4,10,
90                 5,6,11,
91                 7,8,12};
92
93     int c[4][3]={0,0,0,
94                 0,0,0,
95                 0,0,0,
96                 0,0,0};
```

```
97     int m1=4; // number of rows in 1st matrix
98     int n1=4; // number of columns in 1st matrix
99     int n2=3; // number of columns in 2nd matrix
100     int m2=4;
101
102     for(int m=0;m<n2;m++)
103     {
104         for(int j=0;j<m1;j++)
105         {
106             for(int i=0;i<m2;i++)
107             {
108                 *((*(c+j))+m)=*((*(c+j))+m)+*(a[j]+i)*(*(*(b+i)+m));
109             }
110         }
111     }
113     for(int i=0;i<m1;i++)
114     {
115         for(int j=0;j<n2;j++)
116             printf("%d ",c[i][j]);
117
118         printf("\n");
119     }
120 }
```

Create a structure to specify data of customers in a bank. The data to be stored is: Account number, Name, Balance in account. Assume maximum of 200 customers in the bank.

- (a) Write a function to print the Account number and name of each customer with balance below Tk. 100.

```
#include<stdio.h>

struct book{
    char name[25] ;
    int Account_No ;
    int balance;
};

void display(struct book
*b)
{
    for(int i=0;i<200;i++)

    if((b+i)->balance<100)
        printf(" ");
}

int main()
{
    struct book b1[200];
    display(b1);
}
```

Practice Problems

1. Write a function to find the norm of a matrix using pointers. The norm is defined as the square root of the sum of squares of all elements in the matrix.
1. The X and Y coordinates of 10 different points are entered through the keyboard. Write a program to find the distance of last point from the first point (**sum of distance between consecutive points**). [Use pointers]